

COMPTE RENDU DE TP

TP 2 – GreenTechSprint

Active Directory, Wiki.js, VLAN, sauvegarde et automatisation PowerShell

RÉALISÉ PAR

DEHGHAN CYRUS · EN GROUPE DE 4

Groupe BIC2

Ce TP met en place l'infrastructure de l'entreprise **GreenTechSprint**, en deux activités :

- **Activité 1** : création du **domaine Active Directory**, installation du wiki **Wiki.js** sur Debian 12, puis **authentification des utilisateurs par l'AD** ;
- **Activité 2** : **segmentation du réseau en VLAN**, **sauvegarde quotidienne** de la base Wiki.js et **création automatique des comptes** en PowerShell.

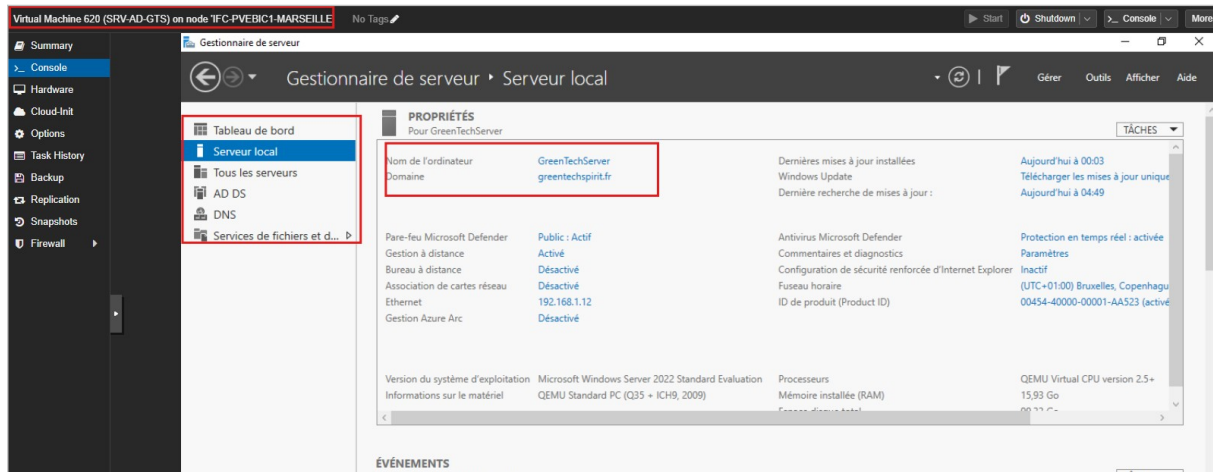
Sommaire

Activité 1 – Mission 1 : Domaine Active Directory.....	3
Activité 1 – Mission 2 : Installation de Wiki.js.....	4
Activité 1 – Mission 3 : Authentification AD sur Wiki.js.....	5
Activité 2 – Mission 1 : Segmentation en VLAN.....	6
Activité 2 – Mission 2 : Sauvegarde de la base Wiki.js.....	10
Activité 2 – Mission 3 : Création des comptes en PowerShell.....	11

Activité 1 – Mission 1 : Domaine Active Directory

Consigne : créer le domaine de l'entreprise **GreenTechSprint.fr**, puis l'installer et le configurer sur **Windows Server 2022**, en respectant la **RFC 1178** pour le nommage du serveur.

Nous avons créé la VM **SRV-AD-GTS** sur le Proxmox et installé **Windows Server 2022**, puis ajouté les rôles **AD DS** et **DNS**. Le serveur, nommé **GreenTechServer** et en adresse fixe **192.168.1.12**, a été promu **contrôleur de domaine** du domaine **greentechspirit.fr** :



Activité 1 – Mission 2 : Installation de Wiki.js

Consigne : pour répondre à la problématique de documentation, installer **Wiki.js** sur une machine **Debian 12 sans interface graphique**.

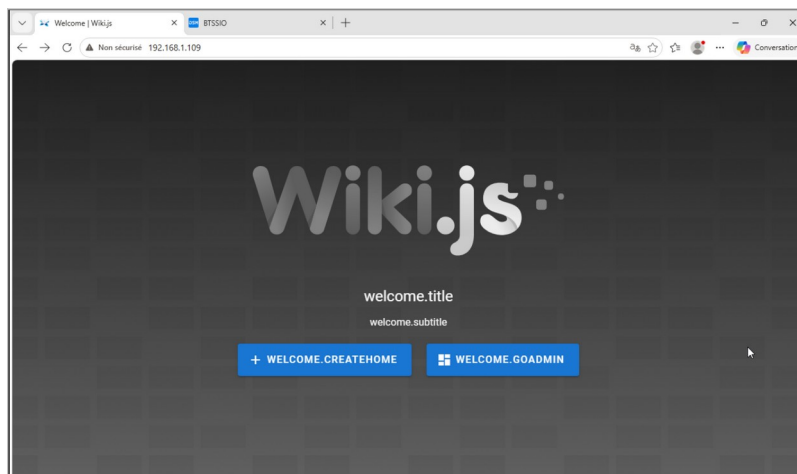
La VM **Debian-WIKIJS** est créée sur le Proxmox à partir de l'ISO **Debian 12 netinst** (2 vCPU, 4 Go de RAM, 32 Go de disque) et installée **sans environnement de bureau** :



Wiki.js est déployé avec **Docker** : un conteneur pour Wiki.js et un pour sa base **PostgreSQL**. La machine a l'adresse **192.168.1.109** :

```
root@debian-wikijs:~# ip -br a
lo                UNKNOWN    127.0.0.1/8 ::1/128
ens18             UP          192.168.1.109/24 fe80::be24:11ff:fe68:9129/64
docker0           DOWN       172.17.0.1/16
br-eedcbf5e327a    UP          172.18.0.1/16 fe80::1407:37ff:fe4d:85ec/64
vethf73027f@if2    UP          fe80::d05e:74ff:fe06:34a/64
veth8ce093a@if2    UP          fe80::7893:ff:fe72:f63a/64
root@debian-wikijs:~#
```

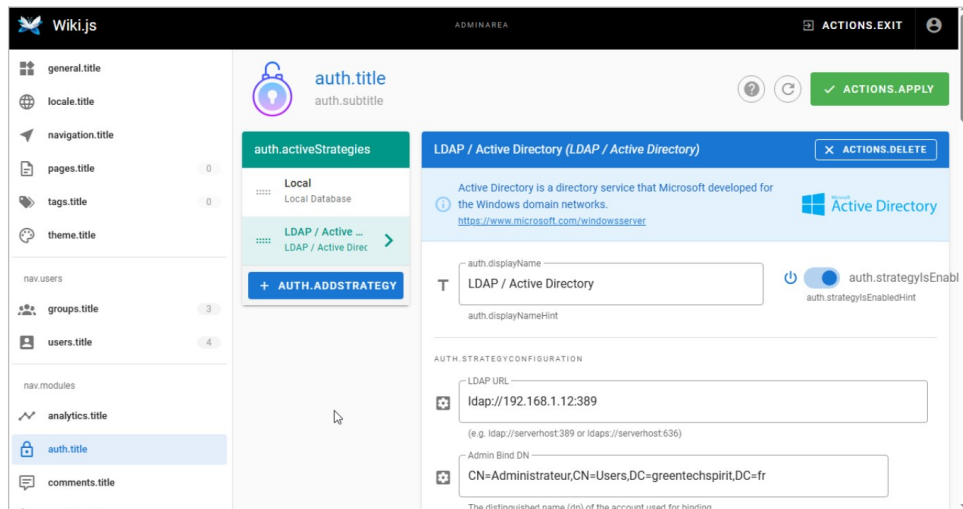
Le wiki est accessible depuis un poste client à l'adresse **http://192.168.1.109** :



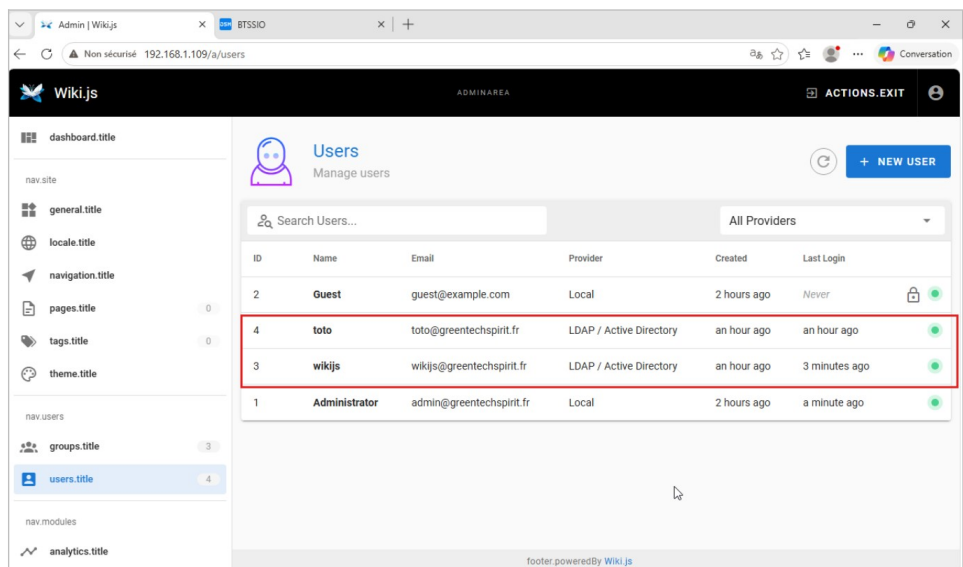
Activité 1 – Mission 3 : Authentification AD sur Wiki.js

Consigne : dans la suite de la centralisation des accès, mettre en œuvre une authentification via l'**Active Directory** depuis le serveur Wiki.js.

Dans l'administration de Wiki.js, nous avons ajouté une stratégie d'authentification **LDAP / Active Directory** reliée au contrôleur de domaine :



Les utilisateurs de test **toto** et **wikijs**, créés dans l'AD, se connectent avec leurs identifiants de domaine et apparaissent avec le fournisseur **LDAP / Active Directory** :

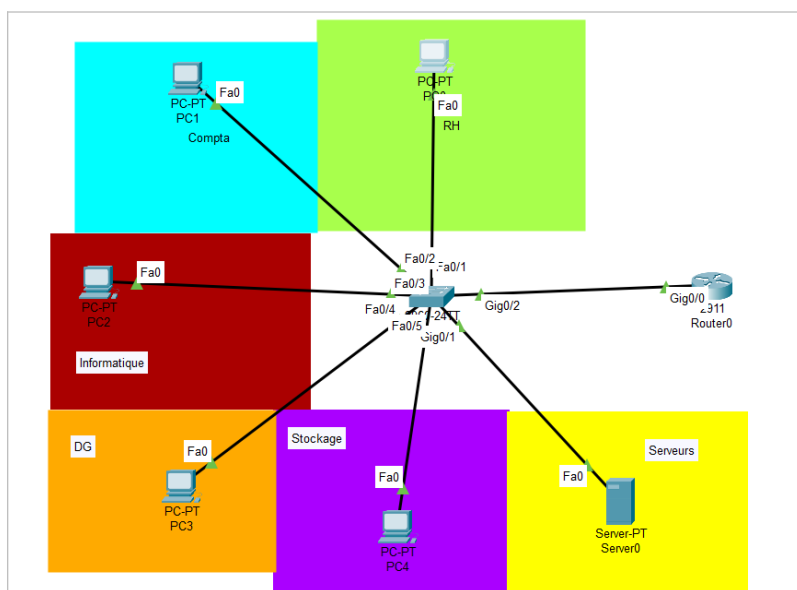


Activité 2 – Mission 1 : Segmentation en VLAN

Consigne : segmenter le réseau en **VLAN** : tous les réseaux représentés, adresses des postes fournies par le **serveur DHCP**, **ping** possible entre toutes les machines (routage inter-VLAN), **dernière IP** de chaque réseau utilisée comme **passerelle**.

Simulation sous **Cisco Packet Tracer** : un switch **2960** (SW1), un routeur **2911** (R1), un poste par service et un serveur DHCP. Chaque service a son VLAN et son réseau en /24, la passerelle est en **.254** :

VLAN	Service	Réseau	Passerelle	Équipement – port
10	Ressources humaines	192.168.10.0/24	192.168.10.254	PC0 – Fa0/1
20	Comptabilité	192.168.20.0/24	192.168.20.254	PC1 – Fa0/2
30	Informatique	192.168.30.0/24	192.168.30.254	PC2 – Fa0/3
40	Direction générale	192.168.40.0/24	192.168.40.254	PC3 – Fa0/4
50	Stockage / reconditionnement	192.168.50.0/24	192.168.50.254	PC4 – Fa0/5
60	Serveurs	192.168.60.0/24	192.168.60.254	Server0 (.60.1) – Gig0/1



Switch : VLAN et trunk

Les six VLAN sont créés et nommés. Chaque port passe en **access** dans le VLAN de son poste ; le port **Gig0/2**, relié au routeur, passe en **trunk** pour transporter tous les VLAN :

```
enable
configure terminal
hostname SW1
vlan 10
 name RH
vlan 20
 name Compta
vlan 30
 name Informatique
vlan 40
 name Direction
vlan 50
 name Stockage
vlan 60
 name Serveurs
exit
```

```
interface fa0/1
 switchport mode access
 switchport access vlan 10
(idem fa0/2 à fa0/5 : VLAN 20 à 50)
interface gig0/1
 switchport mode access
 switchport access vlan 60
interface gig0/2
 switchport mode trunk
end
write memory
```

Vérification avec **show vlan brief** et **show interfaces trunk** :

```
SW1#show vlan brief

VLAN Name                Status    Ports
-----
1    default                active    Fa0/6, Fa0/7, Fa0/8, Fa0/9
                                           Fa0/10, Fa0/11, Fa0/12, Fa0/13
                                           Fa0/14, Fa0/15, Fa0/16, Fa0/17
                                           Fa0/18, Fa0/19, Fa0/20, Fa0/21
                                           Fa0/22, Fa0/23, Fa0/24
10   RH                     active    Fa0/1
20   Compta                 active    Fa0/2
30   Informatique           active    Fa0/3
40   Direction              active    Fa0/4
50   Stockage                active    Fa0/5
60   Serveurs                active    Gig0/1
1002 fddi-default          active
1003 token-ring-default    active
1004 fddinet-default       active
1005 trnet-default         active

SW1#show interfaces trunk

Port      Mode        Encapsulation  Status      Native vlan
Gig0/2    on          802.1q         trunking    1

Port      Vlans allowed on trunk
Gig0/2    1-1005

Port      Vlans allowed and active in management domain
Gig0/2    1,10,20,30,40,50,60

Port      Vlans in spanning tree forwarding state and not pruned
Gig0/2    1,10,20,30,40,50,60

SW1#
```

Routeur : routage inter-VLAN

Méthode **router-on-a-stick** : l'interface **Gig0/0** est découpée en une **sous-interface par VLAN** (encapsulation **802.1Q**), qui porte l'adresse **.254** de la passerelle. La commande **ip helper-address** relaie les demandes DHCP vers le serveur du VLAN 60 :

```
enable
configure terminal
hostname R1
interface gig0/0
  no shutdown
interface gig0/0.10
  encapsulation dot1q 10
  ip address 192.168.10.254 255.255.255.0
  ip helper-address 192.168.60.1

(interface gig0/0.20 à gig0/0.50)

interface gig0/0.60
  encapsulation dot1q 60
  ip address 192.168.60.254 255.255.255.0
end
write memory
```

Les six sous-interfaces sont **up/up** avec leur passerelle en **.254** :

```
R1#show ip interface brief
Interface                IP-Address      OK? Method Status        Protocol
GigabitEthernet0/0       unassigned      YES unset  up            up
GigabitEthernet0/0.10    192.168.10.254  YES manual up            up
GigabitEthernet0/0.20    192.168.20.254  YES manual up            up
GigabitEthernet0/0.30    192.168.30.254  YES manual up            up
GigabitEthernet0/0.40    192.168.40.254  YES manual up            up
GigabitEthernet0/0.50    192.168.50.254  YES manual up            up
GigabitEthernet0/0.60    192.168.60.254  YES manual up            up
GigabitEthernet0/1       unassigned      YES unset  administratively down down
GigabitEthernet0/2       unassigned      YES unset  administratively down down
Vlan1                    unassigned      YES unset  administratively down down
R1#
```

Serveur DHCP

Server0 est en adresse fixe **192.168.60.1** (passerelle 192.168.60.254). Un **pool par VLAN** (VLAN10 à VLAN50) distribue les adresses à partir de **.10**, avec la passerelle **.254** et le DNS **192.168.60.1** :

DHCP

Interface

FastEthernet0

Service

☒ On

☐ Off

Pool Name

serverPool

Default Gateway

0.0.0.0

DNS Server

0.0.0.0

Start IP Address :

192

168

60

0

Subnet Mask:

255

255

255

0

Maximum Number of Users :

512

TFTP Server:

0.0.0.0

WLC Address:

0.0.0.0

Add

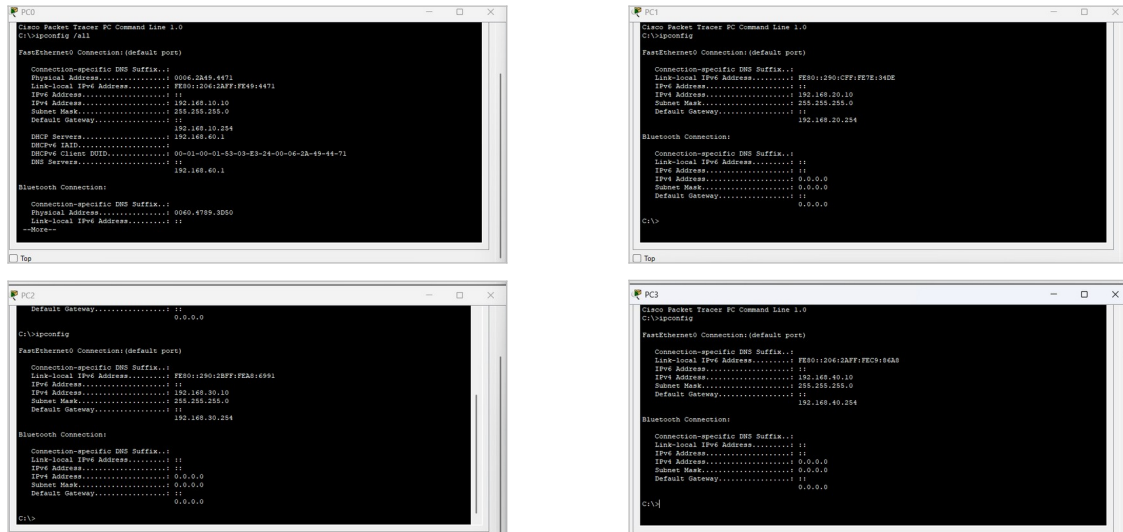
Save

Remove

Pool Name	Default Gateway	DNS Server	Start IP Address	Subnet Mask	Max User	TFTP Server	WLC Address
VLAN50	192.168.50.254	192.168.60.1	192.168.50.10	255.255.255.0	246	0.0.0.0	0.0.0.0
VLAN40	192.168.40.254	192.168.60.1	192.168.40.10	255.255.255.0	246	0.0.0.0	0.0.0.0
VLAN30	192.168.30.254	192.168.60.1	192.168.30.10	255.255.255.0	246	0.0.0.0	0.0.0.0
VLAN10	192.168.10.254	192.168.60.1	192.168.10.10	255.255.255.0	246	0.0.0.0	0.0.0.0
VLAN20	192.168.20.254	192.168.60.1	192.168.20.10	255.255.255.0	246	0.0.0.0	0.0.0.0
serverPool	0.0.0.0	0.0.0.0	192.168.60.0	255.255.255.0	512	0.0.0.0	0.0.0.0

Postes en DHCP

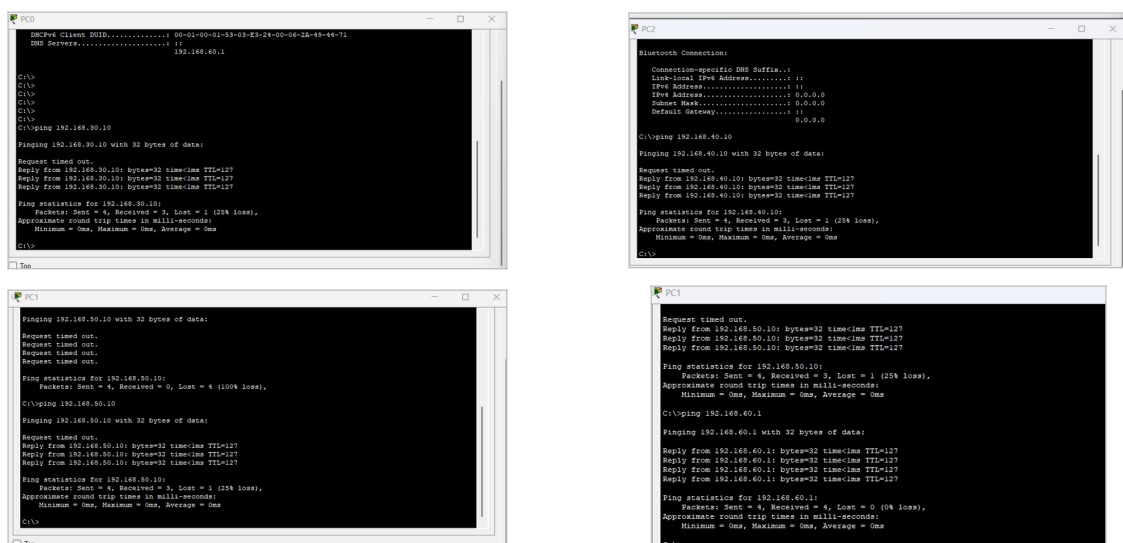
Chaque poste, en **DHCP**, reçoit la première adresse de son pool et la passerelle de son VLAN. Le champ **DHCP Server** de PC0 (192.168.60.1) montre que la réponse traverse bien le routeur :



Tests de connectivité

Les ping entre VLAN différents aboutissent. Le **TTL=127** prouve le passage par le routeur ; le premier paquet perdu correspond à la résolution **ARP** :

Source	Destination	Résultat
PC0 (VLAN 10)	PC2 – 192.168.30.10 (VLAN 30)	3 réponses sur 4
PC2 (VLAN 30)	PC3 – 192.168.40.10 (VLAN 40)	3 réponses sur 4
PC1 (VLAN 20)	PC4 – 192.168.50.10 (VLAN 50)	3 réponses sur 4
PC1 (VLAN 20)	Server0 – 192.168.60.1 (VLAN 60)	4 réponses sur 4



Bilan

Consigne	Preuve
Réseau segmenté en VLAN, tous les réseaux représentés	Schéma, show vlan brief
Adresses distribuées par le serveur DHCP	Pools DHCP, ipconfig des postes
Ping entre toutes les machines (routage inter-VLAN)	Sous-interfaces up/up, tests de ping
Dernière IP de chaque réseau en passerelle	Adresses .254 du routeur et des postes

Activité 2 – Mission 2 : Sauvegarde de la base Wiki.js

Consigne : exporter chaque jour à **23 h** la base de données de Wiki.js dans **/srv/BDD**, via la **crontab** du serveur Debian, sous le nom **BKP_BDD_WIKI_JOUR_MOIS_ANNEE.SQL**.

Le script **/usr/local/bin/backup_wikijs.sh** exporte la base PostgreSQL de Wiki.js dans **/srv/BDD**. Sa planification est ajoutée à la crontab de root (**crontab -e**) :

```
0 23 * * * /usr/local/bin/backup_wikijs.sh
```

Le script s'exécute ainsi à **23 h 00 tous les jours** (minute 0, heure 23, puis trois étoiles pour « tous les jours, tous les mois, toute la semaine »). La commande **crontab -l** confirme la tâche :

```
root@debian-wikijs:~# crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow _ command
0 23 * * * /usr/local/bin/backup_wikijs.sh
root@debian-wikijs: # _
```

L'export est bien créé dans **/srv/BDD**, nommé avec la date du jour :

```
root@debian-wikijs:~# ls -lh /srv/BDD/BKP_BDD_WIKI_*.SQL
-rw-r--r-- 1 root root 57K 7 oct. 13:44 /srv/BDD/BKP_BDD_WIKI_07_10_2026.SQL
root@debian-wikijs:~# echo "BKP_BDD_WIKI_$(date +%d_%m_%Y).SQL"
BKP_BDD_WIKI_07_10_2026.SQL
root@debian-wikijs: #
```

Activité 2 – Mission 3 : Création des comptes en PowerShell

Consigne : écrire un **script PowerShell** qui crée automatiquement les **utilisateurs AD** à partir du **fichier CSV** fourni en annexe 3.

Le script **New-ADUsersFromCsv.ps1** lit le fichier **Utilisateurs.csv** et crée un compte AD pour chaque ligne. Il se lance avec :

```
.\New-ADUsersFromCsv.ps1 -CsvPath C:\Scripts\Utilisateurs.csv
```

Chaque création est confirmée dans la console :

```
PS C:\Scripts> .\New-ADUsersFromCsv.ps1 -CsvPath C:\Scripts\Utilisateurs.csv
[SUCCESS] Utilisateur Hardouin Parizeau (Hardouin_Parizeau) créé avec succès.
[SUCCESS] Utilisateur William Dumont (William_Dumont) créé avec succès.
[SUCCESS] Utilisateur Auda Chicoine (Auda_Chicoine) créé avec succès.
[SUCCESS] Utilisateur Agramant Bolduc (Agramant_Bolduc) créé avec succès.
[SUCCESS] Utilisateur Sacripant Vachon (Sacripant_Vachon) créé avec succès.
[SUCCESS] Utilisateur Luc Louis (Luc_Louis) créé avec succès.
[SUCCESS] Utilisateur Alain Phaneuf (Alain_Phaneuf) créé avec succès.
[SUCCESS] Utilisateur Gabriel Dupont (Gabriel_Dupont) créé avec succès.
[SUCCESS] Utilisateur Nicholas Bizier (Nicholas_Bizier) créé avec succès.
[SUCCESS] Utilisateur Dexter Lebrun (Dexter_Lebrun) créé avec succès.
[SUCCESS] Utilisateur Grosvenor Desrosiers (Grosvenor_Desrosiers) créé avec succès.
[SUCCESS] Utilisateur Stephane Deslauriers (Stephane_Deslauriers) créé avec succès.
[SUCCESS] Utilisateur Anton Masse (Anton_Masse) créé avec succès.
[SUCCESS] Utilisateur Oriol Parenteau (Oriol_Parenteau) créé avec succès.
[SUCCESS] Utilisateur Gallia Sarrazin (Gallia_Sarrazin) créé avec succès.
[SUCCESS] Utilisateur Bruno Cotuand (Bruno_Cotuand) créé avec succès.
[SUCCESS] Utilisateur Rachelle Paimboeuf (Rachelle_Paimboeuf) créé avec succès.
[SUCCESS] Utilisateur Danielle Lafreniere (Danielle_Lafreniere) créé avec succès.
[SUCCESS] Utilisateur Zerbino Theriault (Zerbino_Theriault) créé avec succès.
[SUCCESS] Utilisateur Juliette Dube (Juliette_Dube) créé avec succès.
[SUCCESS] Utilisateur Nicolette Dumoulin (Nicolette_Dumoulin) créé avec succès.
[SUCCESS] Utilisateur Mason Leroy (Mason_Leroy) créé avec succès.
```

Les comptes apparaissent dans **Utilisateurs et ordinateurs Active Directory** :

